Lab 4.1: Adding a Splitter Bar

In this lab, you will add a splitter bar to a single document interface (SDI) application.

To see a demonstration of the lab solution, click this icon.
(CD-ROM plays the demonstration, "Lab 4.1 Demonstration.")

Estimated time to complete this lab: **20 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch04\Lab4.1\Ex01.

The solution code for this lab is located in the folder \Labs\Ch04\Lab4.1\Ex01\Solution.

Objectives

After completing this lab, you will be able to:

- Add a class to an application using ClassWizard.
- Add a static splitter bar to a window to create a split pane.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercise provides practice working with the concepts and techniques covered in this chapter:

Exercise 1: Adding a Splitter Window

In this exercise, you will add a splitter bar to an SDI window.

104 Exercise 1: Adding a Splitter Window

The code that forms the starting point for this exercise is in \Labs\Ch04\Lab4.1\Ex01.

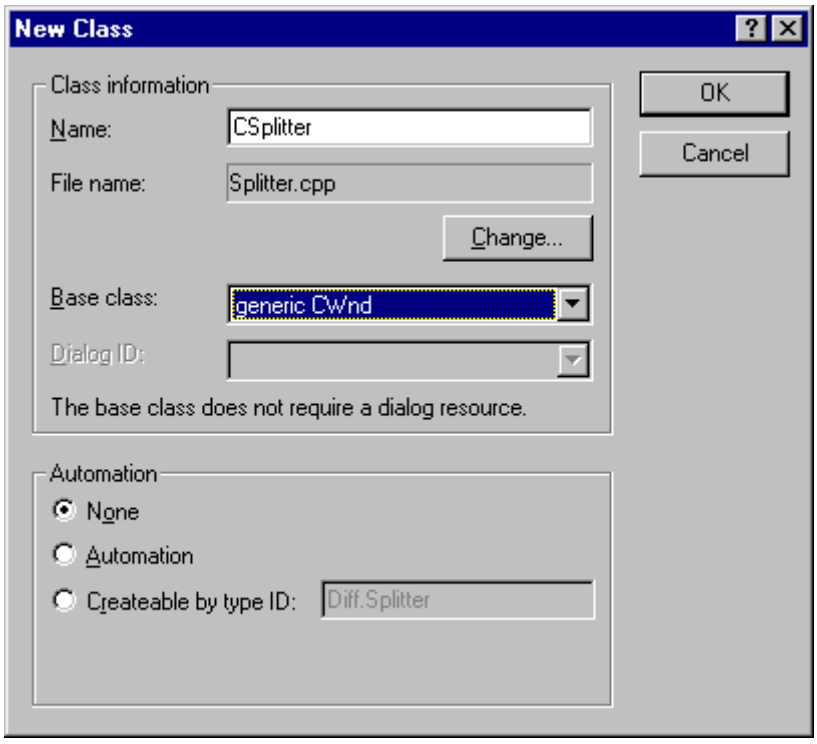
In this exercise, you will add a splitter bar to an SDI application.

▶ **Create a new CSplitter class**

☐ 1. On the **View** menu, click **ClassWizard**.

☐ 2. Click **Add Class**, and then click **New**.

☐ 3. Set the name of the class to **CSplitter**, and base the class on the generic **CWnd** class. Accept the defaults for the other fields. Click **OK**.



Note ClassWizard does not present **CSplitterWnd** as a base class. You can change the base class directly.

☐ 4. Edit the file Splitter.h. Click **CSplitter** in FileView, then click **Go to Definition**.

Change the declaration line of **CSplitter** from:

```
class CSplitter : public CWnd
```

to:

```
class CSplitter : public CSplitterWnd
```

Notes

☐ 5. Add a method to get the protected width of the splitter window to the public attributes section of the **CSplitter** definition as follows:

```
int GetSplitterWidth() const { return m_cxSplitter; }
```

☐ 6. Open the file Splitter.cpp and update the message map declaration. It reads:

```
BEGIN_MESSAGE_MAP(CSplitter, CWnd)
    //{{AFX_MSG_MAP(CSplitter)
```

☐ 7. Change the first line to reference **CSplitterWnd** instead of **CWnd** as follows:

```
BEGIN_MESSAGE_MAP(CSplitter, CSplitterWnd)
```

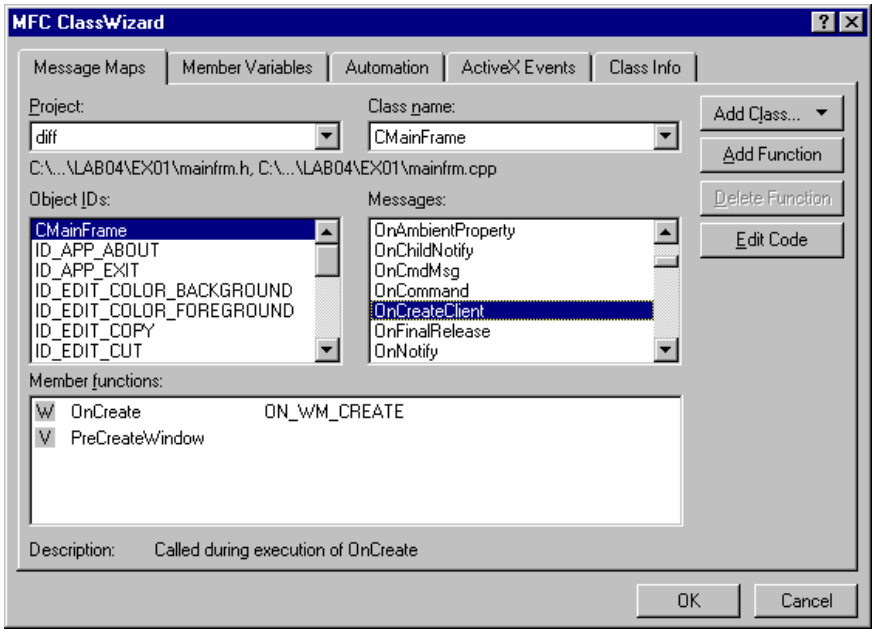
▶ **Add a reference to the splitter in the MainFrame object**

☐ 1. Open the file MainFrm.h.

☐ 2. Add a protected **CSplitter** member to the class definition as follows:

```
// splitter bar embedded members
CSplitter   m_wndSplitter;
```

☐ 3. Use ClassWizard to add an **OnCreateClient** handler. ClassWizard creates a reference to the **OnCreateClient** handler in MainFrm.h and a blank implementation in the file MainFrm.cpp.



The major implementation task in adding a splitter bar to an application is to create the split window itself. Because only two files are allowed to be active, you use static splitter windows. Put the bar in the middle of the frame. For more information, see "Multiple Document Types, Views, and Frame Windows" under "Static splitter windows" in the MSDN Library Visual Studio 6.0.

Notes

▶ **Add code to the OnCreateClient handler to create the splitter window**

☐ 1. Create the static splitter window by adding a call to **CSplitterWnd::CreateStatic** as follows:

```
m_wndSplitter.CreateStatic (this, 1, 2, WS_CHILD);
```

CSplitterWnd::CreateStatic sets up a constant number and arrangement of splitter panes.

☐ 2. Size the splitter windows to two equal panes as follows:

```
SIZE size;
CRect rect;
GetClientRect(&rect);
size.cx = (rect.right - m_wndSplitter.GetSplitterWidth())/2;
size.cy = rect.bottom;
```

☐ 3. Attach the views to the windows as follows:

```
m_wndSplitter.CreateView(0,0,RUNTIME_CLASS(CDiffView), size,    pContext);
m_wndSplitter.CreateView(0,1,RUNTIME_CLASS(CDiffView), size,    pContext);
SetActiveView((CView *)m_wndSplitter.GetPane(0,1));
```

4. Show the splitter window as follows:

```
m_wndSplitter.ShowWindow(SW_SHOWNORMAL);
m_wndSplitter.UpdateWindow();
return TRUE
```

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
BOOL CMainFrame::OnCreateClient(LPCREATESTRUCT /*lpcs*/,
    CCreateContext* pContext)
{
    // Create a static splitter window with two side-by-side panes

    if(!m_wndSplitter.CreateStatic (this,1,2,WS_CHILD))
    {
        return FALSE;
    }

    // Calculate the size of the splitter panes
    SIZE    size;
    CRect   rect;
    GetClientRect(&rect);

    size.cx = (rect.right - m_wndSplitter.GetSplitterWidth())/2;
    size.cy = rect.bottom;
```

(Code continues on following page.)

Notes

```
    //set the views
    m_wndSplitter.CreateView(0,0,RUNTIME_CLASS(CDiffView), size,
        pContext);
    m_wndSplitter.CreateView(0,1,RUNTIME_CLASS(CDiffView), size,
        pContext);
    SetActiveView((CView *)m_wndSplitter.GetPane(0,1));

    //show the splitter
    m_wndSplitter.ShowWindow(SW_SHOWNORMAL);
    m_wndSplitter.UpdateWindow();

    return TRUE;
}
```

▶ **Build, and run the Diff application**

☐ 1. In MainFrm.cpp, include Splitter.h, the file DiffDoc.h., and the file DiffView.h. Splitter.h must be included before MainFrm.h, because there is a **CSplitter** member of **CMainFrame**.

```
#include "splitter.h"
#include "MainFrm.h"
#include "diffdoc.h"
#include "diffview.h"
```

☐ 2. In the file Diff.cpp, include Splitter.h before MainFrm.h

```
#include "splitter.h"
#include "MainFrm.h"
```

☐ 3. Build and run the application. You will notice that you can reposition the splitter bar, but you cannot delete or add splitters.

The solution code for this exercise is located in the folder \Labs\Ch04\Lab4.1\Ex01\Solution.

Notes